Vidyavardhaka Sangha®, Mysore
VIDYAVARDHAKA COLLEGE OF ENGINEERING
Autonomous Institute, Affiliated to Visvesvaraya Technological University, Belagavi
(Approved by AICTE, New Delhi & Government of Karnataka)
Accredited by NBA (CV, CS, EE, EC, IS & ME) | NAAC with 'A' Grade
P.B. No. 206, Gokulam III Stage, Mysuru-570 002, Karnataka, India
Phone: +91 821 4276201 /202 /225, Fax: +91 824 2510677
Web: http://www.vvce.ac.in @vvceofficial

BIG DATA ANALYTICS (BISBD713)

Activity Based Assessment

On

Real-Time UPI Fraud Detection Using Apache Flink

Submitted By

AKANKSH SINGH	**4VV23IS008**
ARYAN SAHA	**4VV23IS027**

Under the guidance of
Prof. Spoorthi M

DEPARTMENT OF INFORMATION SCIENCE AND ENGINEERING
2026–27

1. Introduction to the Tool

Apache Flink is an open-source, distributed engine for stateful computations over data streams. It processes every event the moment it arrives instead of collecting events into batches, so results are available within seconds while the data is still flowing.

Because Flink keeps state for each key, a job can remember what it has seen about a user and use it to judge the next event. Event time keeps results correct when data arrives late, and checkpoints let a job recover from a failure without losing or repeating results.

In this project, FraudStream – Real-Time UPI Fraud Detection is a payment-monitoring demonstration. A simulator generates UPI payments from 300 users in eight Karnataka cities, each with a transaction id, user id, amount, merchant, city and timestamp. FastAPI writes each batch of payments as a JSON-lines file into a shared input folder. Apache Flink picks up each file, validates every record, scores each payment against the user's recent history as normal, suspicious or fraud, counts payments per city in one-minute windows and writes the results to output folders. A watcher in the backend stores the committed results in SQLite, and a dashboard with four tabs presents them. In this design Flink is the only component that decides whether a payment is fraudulent.

2. Description of the Tool: Apache Flink

Apache Flink models a program as a dataflow of sources, operators and sinks. The JobManager turns the dataflow into parallel tasks and runs them in the task slots of one or more TaskManagers, which pass records to each other as streams. FraudStream uses Flink 1.20.1 through its Python DataStream API, PyFlink.

2.1 Components of Apache Flink

- **JobManager:** Coordinates the cluster. It schedules tasks, triggers checkpoints, restarts the job after a failure and serves the Web UI.
- **TaskManager:** A worker process that runs the tasks in its task slots. The FraudStream TaskManager offers four slots.
- **Operator:** One step of the job, such as validating or windowing, run as several parallel subtasks.
- **Keyed state:** Memory kept per key after key_by. All payments of one user reach the same subtask, which holds that user's profile.
- **Watermark:** A marker telling the job that event time has reached a given point, so a time window knows when it is complete.
- **Checkpoint:** A consistent snapshot of every operator's state and the source position, taken every five seconds in this project.

2.2 Features of Apache Flink

- **True streaming:** Each event is processed as it arrives rather than in micro-batches.
- **Stateful processing:** Large per-key state lives inside Flink and is saved with every checkpoint.
- **Event-time windows:** Watermarks and allowed lateness keep per-minute results correct for late data.
- **Exactly-once results:** Checkpoints and transactional sinks make each record affect the output once, even after a crash.
- **Fault tolerance:** A failed TaskManager is replaced and the job resumes from its last checkpoint. The same job runs on a laptop, a standalone cluster, Hadoop YARN or Kubernetes.
- **Monitoring:** The Web UI shows the job graph, records per operator, back pressure and checkpoint history.

3. Operators Used in the Apache Flink Project

The Flink job is written in Python in flink/fraud_job.py and is submitted to the cluster when the stack starts. It uses one FileSource, a Validate function, a keyed FraudDetector, an event-time window and three FileSinks. The source and Validate run as one task so that each user's payments stay in order, and the other operators run with parallelism 2. The configuration below matches Apache Flink 1.20.1 running in Docker with a checkpoint every five seconds, as defined in the project's docker-compose.yml.

3.1 FileSource – Input

FileSource watches the folder /opt/flink/data/incoming inside the container, which is mounted from the project's data/incoming folder on the host, and checks it for new files every second. The simulator writes a hidden temporary file and renames it when complete, so Flink never reads half a file. The list of files already read is saved in each checkpoint, so a file is never read twice.

3.2 Validate – Validation

Validate is a ProcessFunction, chained to the source, that parses each JSON line. It checks that txn_id, user_id, amount, merchant, city and timestamp are present, that the amount is numeric, above zero and within the UPI limit of Rs 1,00,000, and that the timestamp is a valid ISO-8601 date-time. Valid records continue on the main stream. A record that fails is sent to the invalid side output together with its reason.

3.3 FraudDetector – Fraud Scoring

The valid stream is keyed by user_id and passed to FraudDetector, a KeyedProcessFunction that keeps each user's profile in ValueState: the count and total of normal payments and the payments of the last two minutes. An amount over 10 times the user's average is FRAUD, payments from two cities within two minutes are FRAUD, and more than five payments in 60 seconds is SUSPICIOUS. Only non-fraud

payments update the average, so one fraud cannot hide the next.

3.4 Event-Time Window – Aggregation

The scored stream receives watermarks that allow five seconds of out-of-order arrival, is keyed by city and is grouped into one-minute tumbling windows. A ProcessWindowFunction counts normal, suspicious and fraud payments per city and minute, and an allowed lateness of ten minutes lets a late payment update its window.

3.5 FileSink – Output and Monitoring

Three FileSinks write to the scored, invalid and city_minute folders under /opt/flink/data/output, which are mounted from the host so the FastAPI watcher can read them. Each sink commits its file on every checkpoint, so output appears exactly once. The Flink Web UI shows the job graph, the records each operator has handled and every checkpoint, including the one restored after a failure.

4. Execution Procedure

Step 1: Start Apache Flink

Install Docker Desktop with Docker Compose, then build and start the Flink cluster and the backend from the project root.

```
# From the project root
docker compose up -d --build
docker compose ps
```

The first build downloads Apache Flink and PyFlink and can take several minutes. The stack starts a JobManager, a TaskManager, the FastAPI backend and a short-lived job-submit container.

Step 2: Submit the Flink job

The job-submit container waits for the JobManager and then submits flink/fraud_job.py. Follow its log until it reports that the job has been submitted.

```
docker compose logs -f job-submit
```

Open http://localhost:8081. The job FraudStream - UPI Fraud Detection is shown as RUNNING with one TaskManager and four task slots, and a new checkpoint completes every five seconds.

Step 3: Start the backend

The backend runs in its own container and needs no separate setup. To run it outside Docker instead:

```
cd backend
python -m venv venv
source venv/bin/activate          # Windows: venv\Scripts\activate
pip install -r requirements.txt
uvicorn main:app --port 8000
```

The interactive API documentation is available at http://localhost:8000/docs, and http://localhost:8000/api/health returns a JSON status response.

Step 4: Open the dashboard

Open http://localhost:8000. The sidebar shows the backend API and Apache Flink as reachable and the Flink job as RUNNING.

Step 5: Prepare the dataset

An uploaded CSV must include these headers.

txn_id	user_id	amount	merchant	city	timestamp
demo-01	U9001	450	Swiggy	Mysuru	2026-11-03T08:00:00Z
demo-08	U9002	280	Reliance Fresh	Bengaluru	2026-11-03T08:01:20Z
demo-16	U9001	7200	Croma	Mysuru	2026-11-03T08:02:20Z
demo-18	U9004	-350	Amazon	Bengaluru	2026-11-03T08:02:40Z

These rows are taken from sample_transactions.csv, which contains 20 payments; on upload the timestamps are shifted to end at the current time. The rules give 13 normal payments, two suspicious (demo-14 and demo-15), two frauds (demo-08, Mysuru then Bengaluru in 50 seconds, and demo-16, 14.7 times the average) and three rejected records (demo-18, demo-19 and demo-20).

Step 6: Simulate and process

Click Start live stream and wait about two minutes so that Flink builds each user's profile, then click the four fraud scenarios and upload sample_transactions.csv. To show fault tolerance, stop the TaskManager while the stream runs: files wait in data/incoming and the job shows RESTARTING. When it is started again, Flink restores the last checkpoint and processes the waiting files.

```
docker compose stop taskmanager
docker compose start taskmanager
```

Step 7: Verify outputs

- **Alerts:** Confirm that each scenario appears under Latest alerts with the reason Flink gave.
- **Windows:** Check that the Users & cities tab fills one column per minute as watermarks pass.
- **Rejected records:** Check that each bad record is listed with the reason produced by Validate.
- **Exactly-once:** Confirm on the Flink job tab that Duplicate rows stays 0 after the restart.

- **Flink Web UI:** Open the running job and check the records read and written by each operator and the restored checkpoint.

5. Output and Results

The dashboard reports transactions scored, normal, suspicious and fraud payments, the fraud rate, rejected records and Flink's latency, and adds three further tabs: Users & cities, Flink job and Rejected records. The values below were recorded during one test session of about five minutes of live stream at three payments per second, the four fraud scenarios, one upload of sample_transactions.csv and one TaskManager restart.

Metric	Recorded value	Meaning
Transactions scored	976	Payments scored by Flink and stored in SQLite
Normal	963	Payments that matched no fraud rule
Suspicious	8	More than five payments by one user in 60 seconds
Fraud	5	Big-ticket or impossible-travel payments
Rejected	9	Records sent to the invalid side output by Validate
Fraud rate	0.51%	Fraud payments as a share of all scored payments
Flink latency	7.9 s (median)	Time from a payment's timestamp to Flink scoring it
End-to-end latency	11.2 s (median)	Time to reach SQLite, including the checkpoint commit
Checkpoints completed	77	Checkpoints completed by the running job
Recovery	Restored from checkpoint #55	Checkpoint restored after the TaskManager restart
Duplicate rows	0	Rows delivered twice by Flink (exactly-once check)

The sample file gave exactly the expected 13 normal, two suspicious and two fraud payments. After the TaskManager restart Flink restored checkpoint #55, processed the waiting payments and delivered no row twice. Latencies are medians measured before the restart.

5.1 Dashboard tabs

Tab	What it shows
Dashboard	Stream controls, fraud scenarios and CSV upload, summary cards, transactions per minute from the Flink windows, the decision split and the latest alerts with reasons
Users & cities	Top flagged users, amount against the user's average, the city by minute window output and recent transactions with the subtask that scored them
Flink job	Cluster and job state, checkpoints and restores, the duplicate check, operators with records in and out, the subtask split and the progress of every batch
Rejected records	Rejection reasons, the validation rules and every rejected record as raw text

Figure 1. FraudStream dashboard

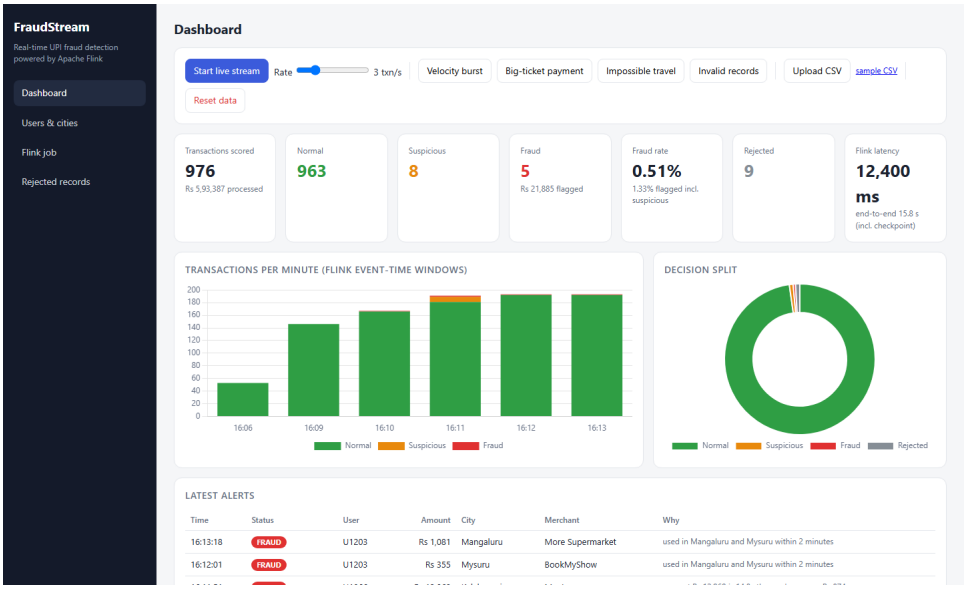


Figure 2. Users & cities tab

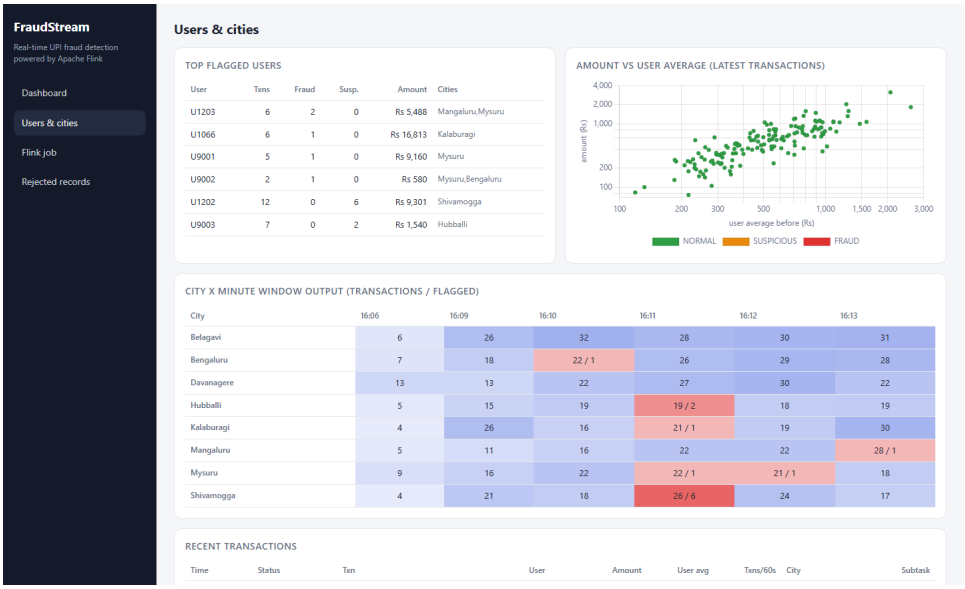


Figure 3. Flink job tab after the TaskManager restart

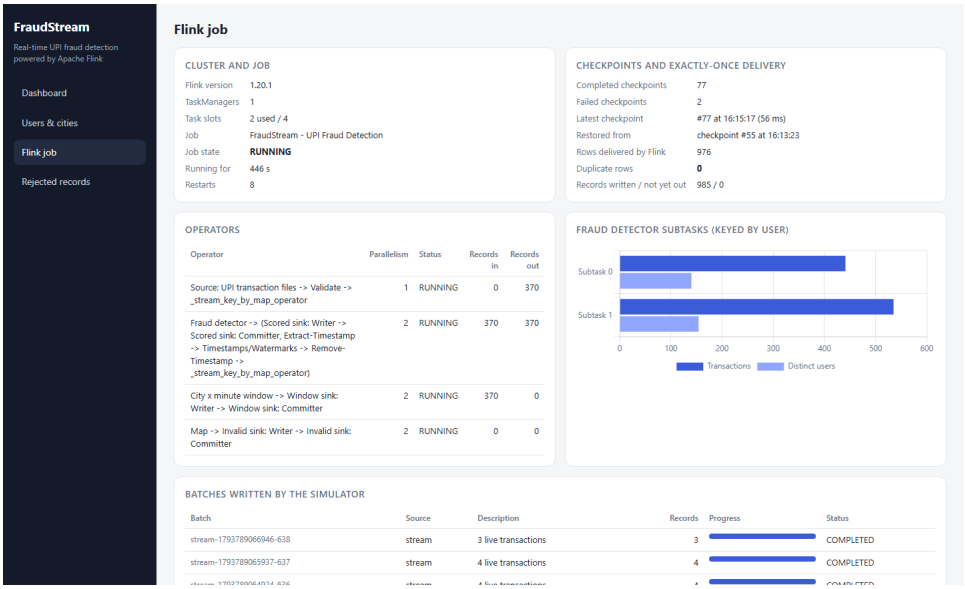
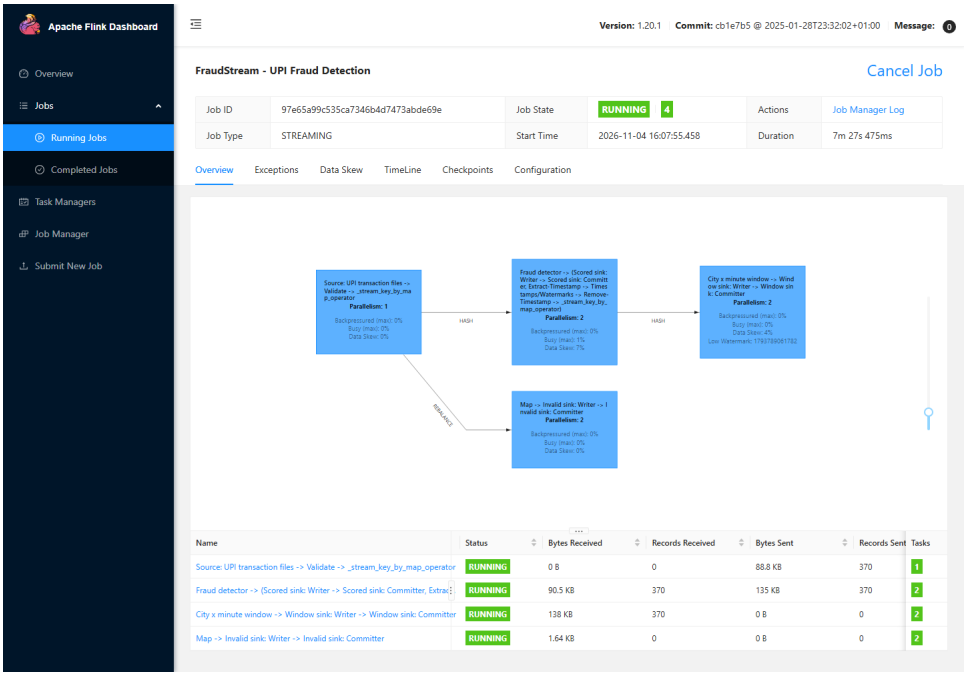


Figure 4. Apache Flink Web UI with the running job



6. Conclusion

FraudStream – Real-Time UPI Fraud Detection demonstrates a complete streaming pipeline built around Apache Flink. Payments are ingested as files, validated with rejects sent to a side output, scored against each user's history held in keyed state, counted per city in event-time windows, written by exactly-once file sinks, stored in SQLite and displayed on a four-tab dashboard. Flink provides the low-latency processing, the state that remembers every user, and the checkpoints that let the job survive a failed TaskManager without losing or repeating a single payment.